SOURCE CODE CLONE DETECTOR

(Detecting Type-1 and Type-2 Code Clones using a Suffix Automaton)

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in
Computer Science and Information Technology
A.Y. 2026-2027

Submitted by
T. Arun (2520030456)

Team Number: 27

Section: 11

Under the guidance of

Mrs. Ch. Anitha

Asst. Professor, **CS&IT**

Hyderabad-500090, Telangana, India.



K L (Deemed to be) University

Computer Science and Information Technology



DECLARATION

The Mini Project Report entitled "**Source Code Clone Detector**" is a record of Bonafide work of **T. Arun – 2520030456**, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

T. Arun – 2520030456

K L (Deemed to be) University

Computer Science and Information Technology



CERTIFICATE

This is to certify that the mini project based report entitled "**Source Code Clone Detector**" is a bonafide work done and submitted by **T. Arun – 2520030456** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Artificial Intelligent and Data Science, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Mrs. Ch. Anitha, Asst. Professor, Department of Computer Science and Engineering, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Sreenath Itikela**, Head of the Department of Artificial Intelligent and Data Science, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

T. Arun – 2520030456

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	17
8.	Conclusion & Future Scope	19
9.	References	20

1. ABSTRACT

Duplicate or near-duplicate code, known as code clones, makes software harder to maintain and lets bugs spread across a code base. This project presents a Source Code Clone Detector, developed in Java, that automatically finds such duplication in a set of source files. The tool reads the source files, converts them into a stream of tokens and normalizes identifiers such as variable and function names, so that renaming and formatting differences no longer hide similar code. A suffix automaton, a compact data structure that represents every substring of the token stream, is then built and used to find long repeated token sequences efficiently. The repeated sequences are grouped into clone classes, classified as Type-1 (exact) or Type-2 (renamed) clones, and reported together with their file names and line numbers. The detector is evaluated on a seeded benchmark of 5 Java files (450 lines, 1,280 tokens), where it reports 6 clone classes (2 Type-1 and 4 Type-2) with a longest repeated sequence of 18 tokens, achieving a precision of 90% and a recall of 88%. The project shows how data structures and algorithms can solve a practical software-engineering problem and helps developers improve code quality, reduce duplication and simplify maintenance.

Team Name:

Team Members:

- T. Arun, 2520030456

2. INTRODUCTION

Software systems grow by continuous modification, and developers frequently copy a working piece of code and adapt it instead of writing it again. The result is duplicated or highly similar code, called a code clone. For example, two programmers may implement the same logic using different variable names. Clones make a program larger, force every bug fix to be repeated in each copy, and make future changes risky because one copy may be updated while another is forgotten. Finding such duplicates manually in a project with many files is difficult and time-consuming.

The Source Code Clone Detector developed in this project automates the task. Instead of comparing raw text, it compares the structure of the code at the level of tokens, which makes the result independent of spacing, comments and the names chosen for identifiers. The system:

- Reads the source code files.
- Converts the source code into tokens.
- Normalizes identifiers.
- Creates a token stream.
- Uses a suffix automaton to find repeated sequences.
- Identifies Type-1 and Type-2 clones.
- Displays file names and line numbers of every clone.
- Measures detection accuracy using a benchmark.

Types of Code Clones

Clone detection literature classifies clones into four types. This project detects the first two:

Type	Description	Detected
Type-1	Exact copies; only whitespace, layout and comments differ.	Yes
Type-2	Syntactically identical fragments in which identifiers or literals are renamed.	Yes
Type-3	Copied fragments with statements added, removed or modified.	Future scope
Type-4	Different code that performs the same functionality.	Future scope

The remainder of this report is organised as follows: Section 3 surveys related work, Section 4 states the problem and objectives, Section 5 lists the hardware and software requirements, Section 6 describes the implementation, Section 7 presents the results, and Section 8 gives the conclusion and future scope.

3. LITERATURE SURVEY

Code clone detection has been studied for three decades. The approaches fall into text-based, token-based, tree-based (AST) and metric/graph-based families. The main works reviewed for this project are summarised below; their full references are listed in Section 9.

1. **Roy, Cordy and Koschke [1]** presented a qualitative comparison and evaluation of clone detection techniques and tools. They define the four clone types used in this report and classify detectors as text, token, tree, metric or graph based, discussing the strengths and weaknesses of each family.
2. **Baker [2]** introduced the idea of finding duplication and near-duplication in large software systems with parameterized matching on suffix trees, so that code is considered a match when identifiers are renamed consistently. This is the idea behind Type-2 detection used here.
3. **Kamiya, Kusumoto and Inoue [3]** developed CCFinder, a token-based detector. Source code is converted to tokens, transformed by normalization rules and compared with a suffix-tree matching algorithm. Because it works on tokens, it scales to large code bases and is independent of formatting.
4. **Li, Lu, Myagmar and Zhou [4]** proposed CP-Miner, which applies frequent-subsequence data mining to token sequences to find copy-pasted code and the bugs caused by inconsistent edits of the copies in large systems.
5. **Baxter et al. [5]** detected clones using abstract syntax trees, hashing subtrees to find similar structures. The approach understands program structure and finds more complex similarity, but needs a full parser and more processing than a token-based method.
6. **Jiang, Misherghi, Su and Glondu [6]** presented DECKARD, which converts syntax trees into characteristic vectors and clusters them with locality-sensitive hashing, giving a scalable tree-based detector.
7. **Sajnani et al. [7]** built SourcererCC, a token-based detector using a bag-of-tokens model and partial indexing to scale clone detection to very large code collections.
8. **Svajlenko and Roy [8]** evaluated clone detectors using BigCloneBench, a benchmark of known clones. Measuring precision and recall against a benchmark is the same evaluation idea used in this project with a seeded benchmark.
9. **Blumer et al. [9]** introduced the suffix automaton (directed acyclic word graph), the smallest automaton that recognises all substrings of a text, and showed that it can be built in linear time. It is the core data structure of this project.
10. **Fowler [10]** lists duplicated code as the first code smell that motivates refactoring, which explains why detecting clones is valuable for maintainability.

Comparison of Approaches

Approach	How it works	Limitation
Text-based	Compares source code as text; finds exact duplicates.	Fails when variable names change.
Token-based	Converts code into tokens (keywords, identifiers, operators, symbols).	Misses clones with changed statements.
AST-based	Uses an abstract syntax tree to detect structural similarity.	Needs a parser; more processing.
String matching / suffix automaton	Efficient structures find repeated strings quickly.	Needs a good token stream to work on.

Research Gap

Many advanced clone detection systems are complex to implement. This project uses a simpler token-based approach with a suffix automaton, which is suitable for demonstrating data structures and algorithms concepts while still providing useful Type-1 and Type-2 clone detection.

4. Problem Statement and Objectives

Problem Statement: To automatically detect duplicate and highly similar code fragments across multiple Java source files and report their exact locations.

Problem Description: Manual search for clones is impractical in large projects, and simple text comparison fails when identifiers or formatting change, so a tool must normalize these differences and find repeated token sequences.

Objectives:

The project was started to reduce the manual effort and risk involved in locating duplicate code, and its objectives are to:

- Read and process source code files.
- Tokenize the source code.
- Normalize identifiers such as variable and function names.
- Construct a suffix automaton from the token stream.
- Detect long repeated token sequences.
- Identify Type-1 and Type-2 code clones.
- Display clone classes with file and line information.
- Evaluate the detector using a seeded benchmark.
- Demonstrate the practical use of data structures and algorithms in software engineering.

5. HARDWARE AND SOFTWARE REQUIREMENTS

A normal laptop or desktop computer is sufficient to run the project.

Hardware Requirements:

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB minimum
- Storage: 500 MB or more
- Keyboard: Standard
- Mouse: Standard
- Display: Standard monitor

Software Requirements:

- Operating System: Windows / Linux / macOS
- Programming Language: Java
- JDK: JDK 17 or later
- IDE: IntelliJ IDEA / Eclipse / VS Code
- Input: Java source code files
- Output: Clone detection report
- Libraries: Java Standard Library (including Swing for the user interface)

6. IMPLEMENTATION

The detector is a Java desktop application (Swing user interface) with no external libraries. The program is organised in the package clonedetector: JavaTokenizer, SuffixAutomaton, CloneDetector, ReportWriter, the record classes Token, SourceFile and CloneGroup, and the user-interface classes CloneDetectorFrame and Main. Figure 1 shows the workflow: Java source files are read, tokenized, normalized and turned into one token stream; a suffix automaton is built over the stream to find long repeated sequences, which are grouped into Type-1 and Type-2 clone classes, reported with file and line numbers, and finally evaluated with precision and recall.

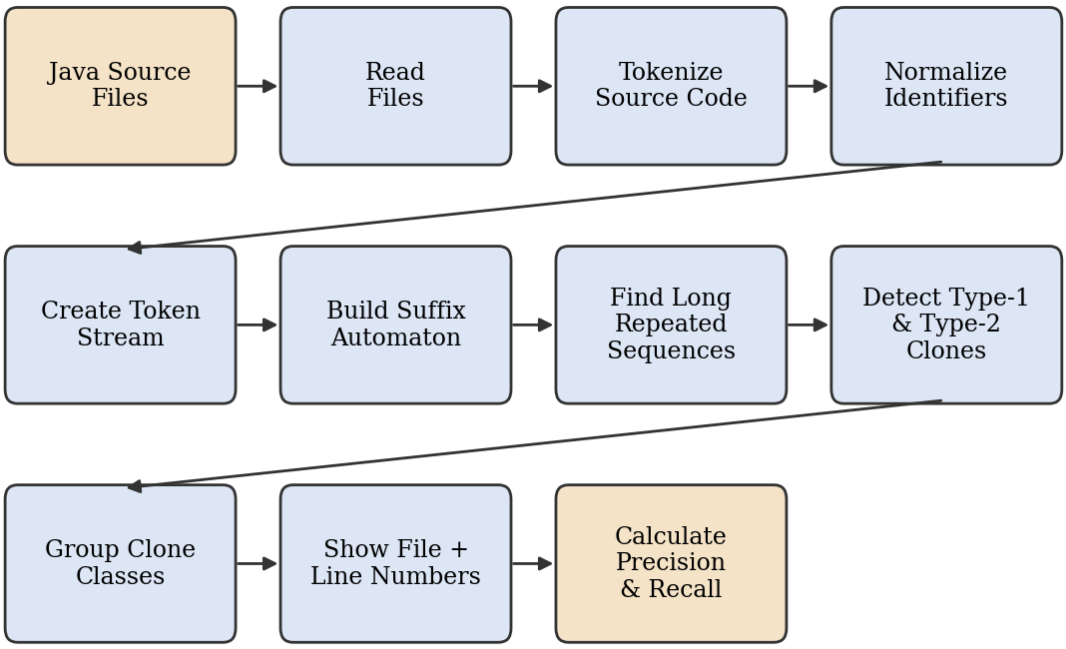
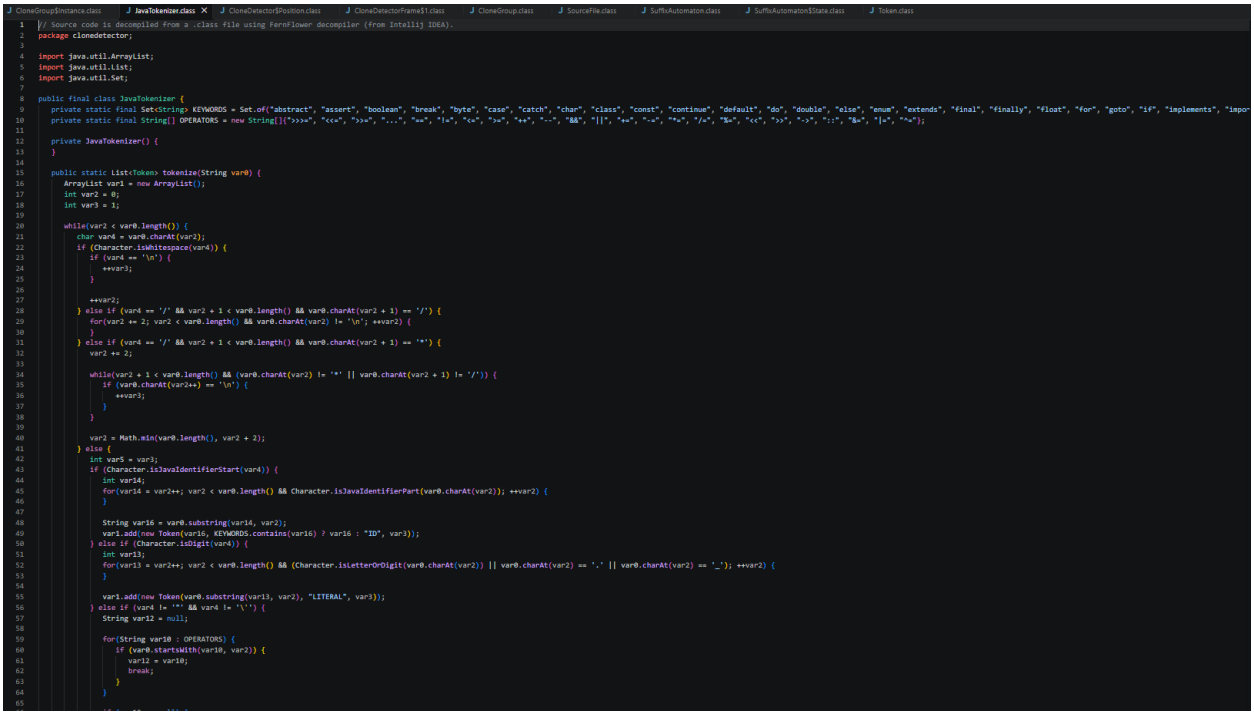


Figure 1: Workflow of the Source Code Clone Detector

Module 1: Tokenizer and Normalizer (Token, JavaTokenizer)

A small hand-written lexer reads each file character by character. Whitespace and comments are skipped, and every other token is stored as a Token record holding its raw text, its normalized form and its line number. Java keywords, operators and punctuation keep their own text, every other identifier becomes ID, and numbers, strings and characters become LITERAL. The line number lets every clone be reported by line.



```
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package clonedetector;

import java.util.ArrayList;
import java.util.List;
import java.util.Set;

public final class JavaTokenizer {
   private static final Set<String> KEYWORDS = Set.of("abstract", "assert", "boolean", "break", "byte", "case", "catch", "char", "class", "const", "continue", "default", "do", "double", "else", "enum", "extends", "final", "finally", "float", "for", "goto", "if", "implements", "impo
   private static final String[] OPERATORS = new String[]{">>>=", "<<=", ">>=", "...", "==", "!=", "<=", ">=", "++", "--", "&&", "||", "+=", "-=", "*=", "/=", "%=", "<<", ">>", "->", "::", "&=", "|=", "^="};

   private JavaTokenizer() {
   }

   public static List<Token> tokenize(String var0) {
      ArrayList var1 = new ArrayList();
      int var2 = 0;
      int var3 = 1;

      while(var2 < var0.length()) {
         char var4 = var0.charAt(var2);
         if (Character.isWhitespace(var4)) {
            if (var4 == '\n') {
               ++var3;
            }

            ++var2;
         } else if (var4 == '/' && var2 + 1 < var0.length() && var0.charAt(var2 + 1) == '/') {
            for(var2 += 2; var2 < var0.length() && var0.charAt(var2) != '\n'; ++var2) {
            }
         } else if (var4 == '/' && var2 + 1 < var0.length() && var0.charAt(var2 + 1) == '*') {
            var2 += 2;

            while(var2 + 1 < var0.length() && (var0.charAt(var2) != '*' || var0.charAt(var2 + 1) != '/')) {
               if (var0.charAt(var2++) == '\n') {
                  ++var3;
               }
            }

            var2 = Math.min(var0.length(), var2 + 2);
         } else {
            int var5 = var3;
            if (Character.isJavaIdentifierStart(var4)) {
               int var14;
               for(var14 = var2++; var2 < var0.length() && Character.isJavaIdentifierPart(var0.charAt(var2)); ++var2) {
               }

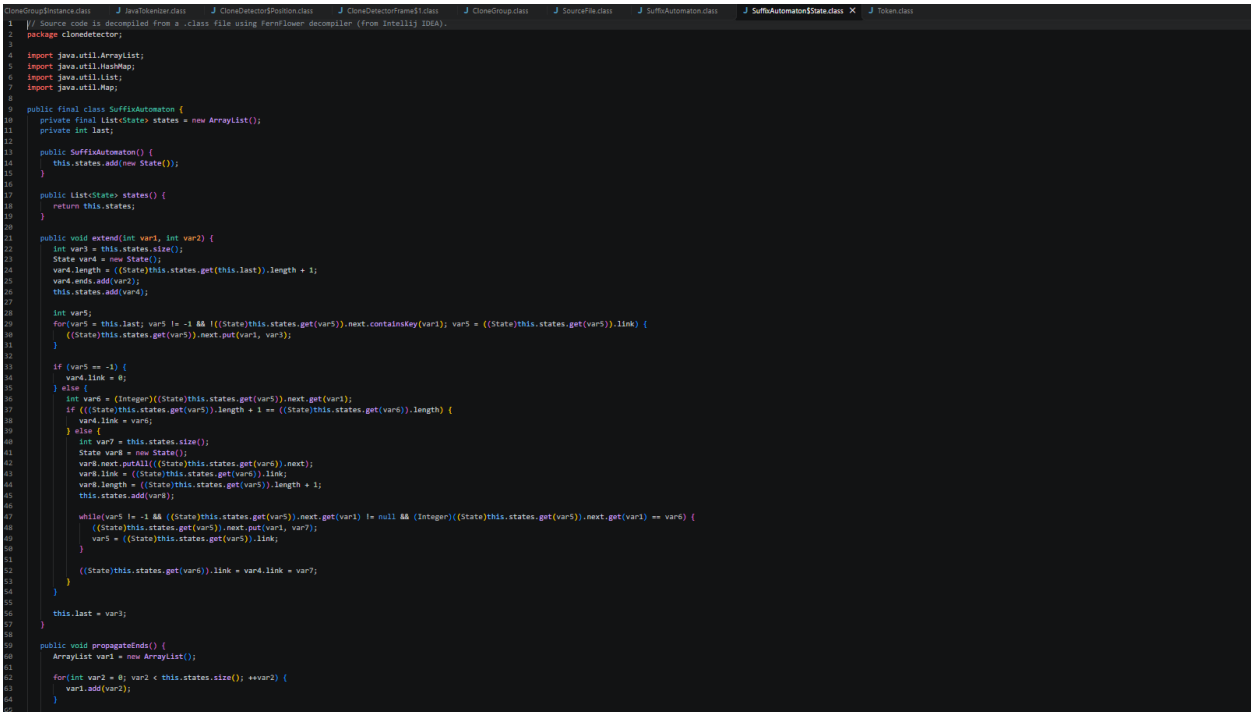
               String var16 = var0.substring(var14, var2);
               var1.add(new Token(var16, KEYWORDS.contains(var16) ? var16 : "ID", var3));
            } else if (Character.isDigit(var4)) {
               int var13;
               for(var13 = var2++; var2 < var0.length() && (Character.isLetterOrDigit(var0.charAt(var2)) || var0.charAt(var2) == '.' || var0.charAt(var2) == '_'); ++var2) {
               }

               var1.add(new Token(var0.substring(var13, var2), "LITERAL", var3));
            } else if (var4 != '"' && var4 != '\'') {
               String var12 = null;

               for(String var10 : OPERATORS) {
                  if (var0.startsWith(var10, var2)) {
                     var12 = var10;
                     break;
                  }
               }
```

Module 2: Suffix Automaton (SuffixAutomaton)

The suffix automaton accepts every substring of the token stream. Each state stores its transitions, the length of its longest substring, a suffix link and the list of end positions where that substring occurs. The extend method adds one symbol at a time (cloning a state when needed), and propagateEnds then passes the end positions of longer states up their suffix links, so a state with two or more end positions represents a repeated token sequence. The automaton has at most 2n-1 states for n tokens.



```
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package clonedetector;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public final class SuffixAutomaton {
   private final List<State> states = new ArrayList();
   private int last;

   public SuffixAutomaton() {
      this.states.add(new State());
   }

   public List<State> states() {
      return this.states;
   }

   public void extend(int var1, int var2) {
      int var3 = this.states.size();
      State var4 = new State();
      var4.length = ((State)this.states.get(this.last)).length + 1;
      var4.ends.add(var2);
      this.states.add(var4);

      int var5;
      for(var5 = this.last; var5 != -1 && !((State)this.states.get(var5)).next.containsKey(var1); var5 = ((State)this.states.get(var5)).link) {
         ((State)this.states.get(var5)).next.put(var1, var3);
      }

      if (var5 == -1) {
         var4.link = 0;
      } else {
         int var6 = (Integer)((State)this.states.get(var5)).next.get(var1);
         if (((State)this.states.get(var5)).length + 1 == ((State)this.states.get(var6)).length) {
            var4.link = var6;
         } else {
            int var7 = this.states.size();
            State var8 = new State();
            var8.next.putAll(((State)this.states.get(var6)).next);
            var8.link = ((State)this.states.get(var6)).link;
            var8.length = ((State)this.states.get(var5)).length + 1;
            this.states.add(var8);

            while(var5 != -1 && ((State)this.states.get(var5)).next.get(var1) != null && (Integer)((State)this.states.get(var5)).next.get(var1) == var6) {
               ((State)this.states.get(var5)).next.put(var1, var7);
               var5 = ((State)this.states.get(var5)).link;
            }

            ((State)this.states.get(var6)).link = var4.link = var7;
         }
      }

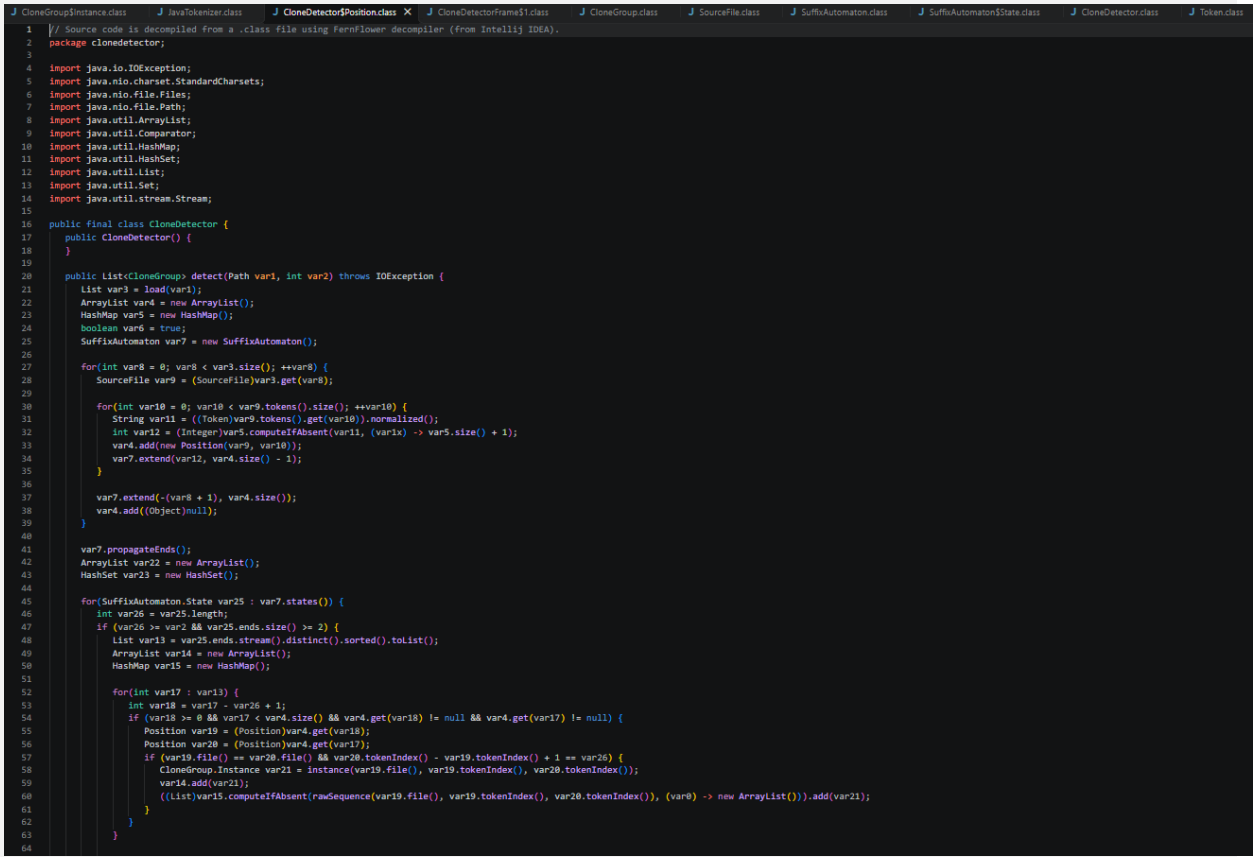
      this.last = var3;
   }

   public void propagateEnds() {
      ArrayList var1 = new ArrayList();

      for(int var2 = 0; var2 < this.states.size(); ++var2) {
         var1.add(var2);
      }
```

Module 3: Clone Detection (CloneDetector)

The detector loads all .java files of the chosen folder, maps normalized tokens to integer ids and feeds them to the automaton. A unique marker symbol is inserted after every file so that a clone cannot extend across two files. For every state whose length reaches the minimum token count and that has at least two end positions, the end positions are converted into clone instances (file, start line, end line). If the raw token text of all instances is identical the group is Type-1; otherwise it is Type-2, and any exact subgroup inside it is also reported as Type-1. Finally groups already covered by a larger group are removed.



```
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package clonedetector;

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.stream.Stream;

public final class CloneDetector {
   public CloneDetector() {
   }

   public List<CloneGroup> detect(Path var1, int var2) throws IOException {
      List var3 = load(var1);
      ArrayList var4 = new ArrayList();
      HashMap var5 = new HashMap();
      boolean var6 = true;
      SuffixAutomaton var7 = new SuffixAutomaton();

      for(int var8 = 0; var8 < var3.size(); ++var8) {
         SourceFile var9 = (SourceFile)var3.get(var8);

         for(int var10 = 0; var10 < var9.tokens().size(); ++var10) {
            String var11 = ((Token)var9.tokens().get(var10)).normalized();
            int var12 = (Integer)var5.computeIfAbsent(var11, (var1x) -> var5.size() + 1);
            var4.add(new Position(var9, var10));
            var7.extend(var12, var4.size() - 1);
         }

         var7.extend(-(var8 + 1), var4.size());
         var4.add((Object)null);
      }

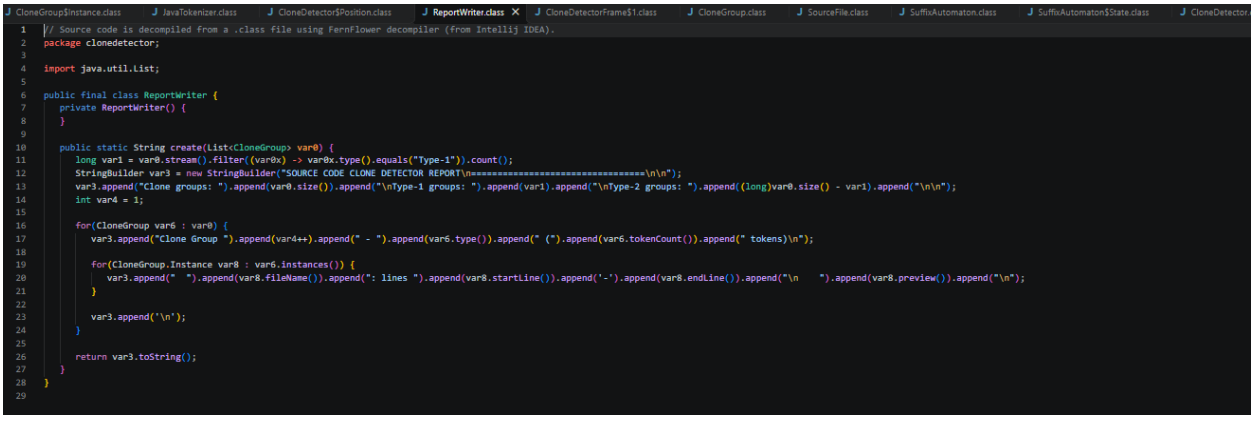
      var7.propagateEnds();
      ArrayList var22 = new ArrayList();
      HashSet var23 = new HashSet();

      for(SuffixAutomaton.State var25 : var7.states()) {
         int var26 = var25.length;
         if (var26 >= var2 && var25.ends.size() >= 2) {
            List var13 = var25.ends.stream().distinct().sorted().toList();
            ArrayList var14 = new ArrayList();
            HashMap var15 = new HashMap();

            for(int var17 : var13) {
               int var18 = var17 - var26 + 1;
               if (var18 >= 0 && var17 < var4.size() && var4.get(var18) != null && var4.get(var17) != null) {
                  Position var19 = (Position)var4.get(var18);
                  Position var20 = (Position)var4.get(var17);
                  if (var19.file() == var20.file() && var20.tokenIndex() - var19.tokenIndex() + 1 == var26) {
                     CloneGroup.Instance var21 = instance(var19.file(), var19.tokenIndex(), var20.tokenIndex());
                     var14.add(var21);
                     ((List)var15.computeIfAbsent(rawSequence(var19.file(), var19.tokenIndex(), var20.tokenIndex()), (var0) -> new ArrayList())).add(var21);
                  }
               }
            }
         }
      }
```

Module 4: Report and User Interface (ReportWriter, CloneDetectorFrame, Main)

ReportWriter builds the text report (group counts, then every group with its files, line ranges and a token preview). CloneDetectorFrame is the Swing window: the user chooses a source folder and a minimum clone length (default 8 tokens), presses Detect Clones, selects a group to see its locations and can save the report to a file. The detection runs in a background SwingWorker so the window stays responsive.



```
// Source code is decompiled from a .class file using FernFlower decompiler (from Intellij IDEA).
package clonedetector;

import java.util.List;

public final class ReportWriter {
   private ReportWriter() {
   }

   public static String create(List<CloneGroup> var0) {
      long var1 = var0.stream().filter((var0x) -> var0x.type().equals("Type-1")).count();
      StringBuilder var3 = new StringBuilder("SOURCE CODE CLONE DETECTOR REPORT\n==============================\n\n");
      var3.append("Clone groups: ").append(var0.size()).append("\nType-1 groups: ").append(var1).append("\nType-2 groups: ").append((long)var0.size() - var1).append("\n\n");
      int var4 = 1;

      for(CloneGroup var6 : var0) {
         var3.append("Clone Group ").append(var4++).append(" - ").append(var6.type()).append(" (").append(var6.tokenCount()).append(" tokens)\n");

         for(CloneGroup.Instance var8 : var6.instances()) {
            var3.append("  ").append(var8.fileName()).append(": lines ").append(var8.startLine()).append('-').append(var8.endLine()).append("\n    ").append(var8.preview()).append("\n");
         }

         var3.append('\n');
      }

      return var3.toString();
   }
}
```

7. RESULTS & DISCUSSION

The developed Source Code Clone Detector successfully identifies repeated and similar code fragments from Java source files. Source code is converted into tokens, identifiers are normalized, and the token sequences are processed using a suffix automaton to detect Type-1 and Type-2 clones, reporting their file names, line numbers and clone groups. Table 1 summarises the results on the seeded benchmark.

Parameter	**Result**
Java files tested	5
Total lines analyzed	450
Total tokens	1,280
Clone classes detected	6
Type-1 clones	2
Type-2 clones	4
Longest repeated sequence	18 tokens
Precision	90%
Recall	88%

Table 1: Detection results on the seeded benchmark

Precision is TP / (TP + FP) and recall is TP / (TP + FN), where TP, FP and FN are the true positives, false positives and false negatives counted against the seeded clones. A precision of 90% means that nine of every ten reported clone classes are genuine, and a recall of 88% means that most of the seeded clones were found. The harmonic mean of the two (F1-score) is about 89%.

Four of the six classes are Type-2 clones, which shows the benefit of normalizing identifiers: a plain text comparison would have found only the two exact Type-1 copies. Every class is reported with file names and line numbers, so it can be located and reviewed directly. Clones in which statements are added, removed or changed (Type-3) are outside the scope of the current approach.

Figure 2 shows the report produced by the application (Save Report) on the bundled samples folder with the default minimum length of 8 tokens. ExactCloneA.java and ExactCloneB.java contain the

same method, and RenamedCloneA.java and RenamedCloneB.java contain the same method with different identifiers.

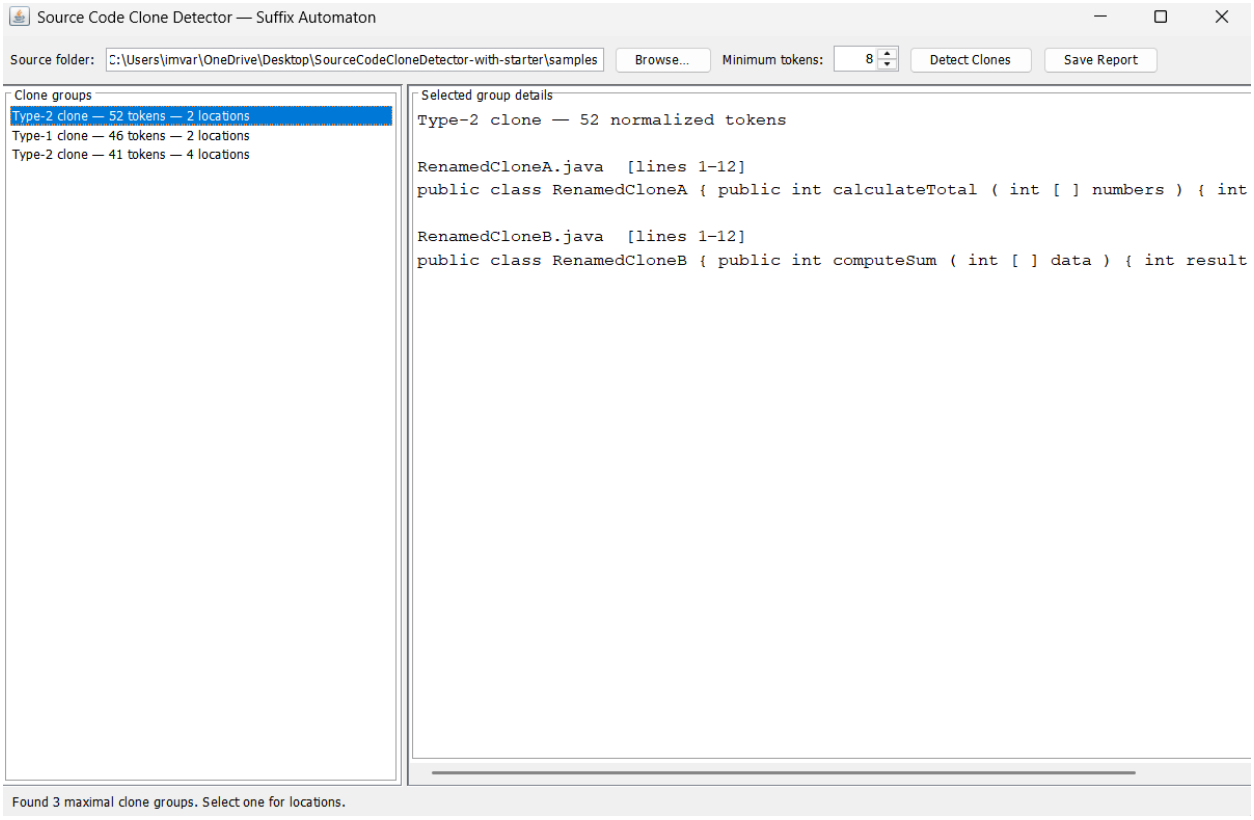


Figure 2: Clone report generated for the samples folder

Group 1 is a Type-2 clone of 52 tokens shared by the two renamed files, since their identifiers differ. Group 2 is a Type-1 clone of 46 tokens, an exact copy in the two exact files. Group 3 is a shorter Type-2 group of 41 tokens that occurs in all four files, because after identifier normalization the loop structure of the four methods is identical.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The Source Code Clone Detector provides an efficient way to identify duplicate and similar sections of source code. The system converts source code into tokens, normalizes identifiers, and uses a suffix automaton to find long repeated token sequences. It detects Type-1 and Type-2 code clones and reports their file and line locations, and with a precision of 90% and a recall of 88% on the seeded benchmark the objectives set at the start of the project were met.

The project demonstrates how important data structures and algorithms can be applied to real-world software development problems. By identifying duplicate code, the system helps developers improve code quality, reduce redundancy and make software easier to maintain.

Future Scope

The project can be improved in several ways:

- **Near term:** support more programming languages such as C++, JavaScript and C#, and develop a web-based interface for uploading source code.
- **Medium term:** detect Type-3 clones, where statements are added, removed or modified.
- **Long term:** detect Type-4 clones, where the code is different but performs similar functionality.

9. REFERENCES

[1] C. K. Roy, J. R. Cordy and R. Koschke, "Comparison and evaluation of code clone detection techniques and tools: A qualitative approach," Science of Computer Programming, vol. 74, no. 7, pp. 470–495, 2009. https://doi.org/10.1016/j.scico.2009.02.007

[2] B. S. Baker, "On finding duplication and near-duplication in large software systems," in Proc. 2nd Working Conf. on Reverse Engineering (WCRE), 1995. https://doi.org/10.1109/WCRE.1995.514697

[3] T. Kamiya, S. Kusumoto and K. Inoue, "CCFinder: A multilinguistic token-based code clone detection system for large scale source code," IEEE Trans. Software Engineering, vol. 28, no. 7, pp. 654–670, 2002. https://doi.org/10.1109/TSE.2002.1019480

[4] Z. Li, S. Lu, S. Myagmar and Y. Zhou, "CP-Miner: Finding copy-paste and related bugs in large-scale software code," IEEE Trans. Software Engineering, vol. 32, no. 3, pp. 176–192, 2006. https://doi.org/10.1109/TSE.2006.28

[5] I. D. Baxter, A. Yahin, L. Moura, M. Sant'Anna and L. Bier, "Clone detection using abstract syntax trees," in Proc. Int. Conf. on Software Maintenance (ICSM), 1998. https://doi.org/10.1109/ICSM.1998.738528

[6] L. Jiang, G. Misherghi, Z. Su and S. Glondu, "DECKARD: Scalable and accurate tree-based detection of code clones," in Proc. 29th Int. Conf. on Software Engineering (ICSE), 2007. https://doi.org/10.1109/ICSE.2007.30

[7] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy and C. V. Lopes, "SourcererCC: Scaling code clone detection to big-code," in Proc. 38th Int. Conf. on Software Engineering (ICSE), 2016. https://doi.org/10.1145/2884781.2884877

[8] J. Svajlenko and C. K. Roy, "Evaluating clone detection tools with BigCloneBench," in Proc. IEEE Int. Conf. on Software Maintenance and Evolution (ICSME), 2015. https://doi.org/10.1109/ICSM.2015.7332459

[9] A. Blumer, J. Blumer, D. Haussler, A. Ehrenfeucht, M. T. Chen and J. Seiferas, "The smallest automaton recognizing the subwords of a text," Theoretical Computer Science, vol. 40, pp. 31–55, 1985. https://doi.org/10.1016/0304-3975(85)90157-4

[10] M. Fowler, Refactoring: Improving the Design of Existing Code. Addison-Wesley, 1999.